로그라이크 강화학습 프로젝트

AI가 스스로 게임을 배우고, 전술을 추천하기까지

3개월 설계 문서 · 입문자용

작성일: 2026-07-01

목차

0. 이 문서를 읽는 법

이 문서는 Python을 이제 막 시작한 입문자를 기준으로 작성했습니다. 어려운 용어는 처음 나올 때 쉬운 말로 풀어서 설명하고, 각 단계마다 '지금 무엇을, 왜 하는지'를 다시 짚어줍니다. 순서대로 읽되, 막히면 언제든지 앞 장으로 돌아와도 됩니다.

전체 구조는 이렇습니다. 먼저 프로젝트가 무엇인지(1장)와 강화학습 개념(2장)을 이해하고, 만들 게임(3장)과 사용할 도구(4장)를 정한 뒤, 전체 그림(5장)을 봅니다. 그다음 3개월을 주 단위로 나눈 실행 계획(6장)을 따라가고, 마지막에 전술 추천 기능(7장)과 위험 요소(8장), 성공 기준(9장), 다음 단계(10장)를 확인합니다.

핵심 한 줄 요약: '작게 만들어서 → AI에게 시켜보고 → 잘 되면 조금씩 키운다'. 이 원칙만 지키면 3개월 안에 충분히 완성할 수 있습니다.

1. 프로젝트 개요

1.1 한 문장 목표

직접 만든 간단한 로그라이크 게임을 AI에게 반복해서 시키고, AI가 강화학습으로 최종 보스를 안정적으로 처치하는 방법을 스스로 익히게 한 뒤, 그렇게 배운 전술을 사람 플레이어에게 상황에 맞게 추천하는 것.

1.2 로그라이크가 뭔가요

로그라이크는 매번 지도와 아이템이 새로 바뀌고, 죽으면 처음부터 다시 시작하는 장르의 게임입니다. 대표적으로 '한 번 죽으면 캐릭터가 사라진다(영구 죽음)', '무작위로 생성되는 던전', '수많은 선택지(어떤 아이템을 먹을지, 어느 길로 갈지, 싸울지 도망칠지)'가 특징입니다. 이 '무작위성'과 '선택의 연속'이 강화학습을 보여주기에 아주 좋은 무대입니다. 왜냐하면 AI가 외운 답을 반복하는 게 아니라, 매번 달라지는 상황에서 판단하는 법을 배워야 하기 때문입니다.

1.3 왜 이 프로젝트가 강화학습을 잘 보여주나

- 선택의 연속: 로그라이크는 매 턴이 선택입니다. 강화학습은 바로 이 '연속된 선택으로 목표를 달성하는 법'을 배우는 기술입니다.

- 명확한 목표: '최종 보스 처치'라는 뚜렷한 목표가 있어, AI가 잘하고 있는지 점수로 측정하기 쉽습니다.
- 눈에 보이는 성장: 처음엔 초반에 죽던 AI가 점점 더 깊이 들어가고, 결국 보스를 잡는 과정을 그래프와 영상으로 보여줄 수 있습니다. '학습했다'는 걸 직관적으로 증명합니다.

1.4 최종 산출물(이 프로젝트가 끝났을 때 손에 남는 것)

- 직접 조작도 가능하고 AI도 플레이할 수 있는 간단한 로그라이크 게임
- 게임을 학습한 AI 모델 파일(보스를 처치할 수 있는 수준)
- 학습 과정 그래프(점수가 오르는 곡선)와 플레이 영상/GIF
- 상황을 입력하면 '이럴 땐 이렇게 하세요'를 알려주는 전술 추천 기능

2. 강화학습 기초 (입문자용)

이 장은 개념만 다룹니다. 코드가 아니라 '무슨 일이 벌어지는지'를 먼저 그림으로 이해하는 게 목적입니다.

2.1 핵심 개념 다섯 가지

강화학습은 강아지 훈련과 비슷합니다. 강아지(AI)에게 명령을 주고, 잘하면 간식(보상)을 줍니다. 강아지는 어떤 행동이 간식을 주는지 시행착오로 배웁니다. 강화학습의 다섯 가지 용어를 이 비유로 정리하면 다음과 같습니다.

용어	쉬운 뜻	우리 게임에서는
에이전트(Agent)	배우는 주체	게임을 플레이하는 AI
환경(Environment)	AI가 활동하는 세계	우리가 만든 로그라이크 게임
상태(State)	지금 상황 정보	체력, 위치, 적 위치, 가진 아이템 등
행동(Action)	AI가 고르는 선택	상하좌우 이동, 공격, 아이템 사용 등
보상(Reward)	잘했는지 알려주는 점수	적 처치 +점수, 피해 -점수, 보스 처치 +큰 점수

2.2 학습이 일어나는 반복 고리

강화학습은 아래 네 단계를 수백만 번 반복하면서 이루어집니다. 사람이 답을 알려주는 게 아니라, AI가 직접 해보고 점수를 보며 스스로 고쳐나갑니다.

1. AI가 현재 상태(체력, 적 위치 등)를 봅니다.
2. 가능한 행동 중 하나를 고릅니다(처음엔 거의 무작위).
3. 게임이 바뀌고, 보상 점수를 받습니다.
4. '이 상황에서 이 행동이 좋았다/나빴다'를 조금씩 학습하고, 1번으로 돌아갑니다.

2.3 왜 '보상 설계'가 가장 중요한가

AI는 오직 보상 점수를 최대로 만들려고만 합니다. 그래서 보상을 잘못 설계하면 엉뚱한 걸 배웁니다. 예를 들어 '오래 살아남으면 +점수'만 주면, AI는 보스에게 다가가지 않고 구석에서 시간만 끌 수도 있습니다. 이 프로젝트에서 가장 많이 고민하고 여러 번 다듬어야 할 부분이 바로 이 보상 설계입니다. 이 문서 6장 로드맵에서도 보상 조정에 시간을 넉넉히 배정했습니다.

2.4 이 프로젝트에서 쓸 알고리즘: PPO

강화학습에는 여러 방법(알고리즘)이 있습니다. 우리는 PPO(Proximal Policy Optimization)를 사용합니다. 이름은 어렵지만, 입문자가 알아야 할 것은 세 가지뿐입니다. 첫째, PPO는 현재 가장 널리 쓰이고 안정적인 방법이라 자료가 많습니다. 둘째, 뒤에 소개할 도구(Stable-Baselines3)에 이미 완성된 형태로 들어 있어 우리가 알고리즘을 직접 구현할 필요가 없습니다. 셋째, 우리는 알고리즘 내부를 만들 필요 없이 '게임(환경)'과 '보상'만 잘 만들면 됩니다. 알고리즘의 수학은 나중에 여유가 생기면 공부해도 늦지 않습니다.

3. 게임 설계 (최대한 단순하게)

입문자 프로젝트의 성패는 '얼마나 작게 시작하느냐'에 달려 있습니다. 화려한 그래픽이나 복잡한 규칙은 3개월 안에 학습까지 끝내는 데 방해가 됩니다. 그래서 처음엔 글자로 표현되는 격자(그리드) 던전으로 시작합니다.

3.1 최소 버전(MVP) 규칙

요소	설계
맵	예: 9x9 또는 11x11 칸으로 이루어진 격자. 벽(#), 빈 칸(.), 시작점, 출구
플레이어	체력(HP), 공격력, 위치를 가짐. 상/하/좌/우 이동과 공격 가능
적	약한 몬스터 몇 마리. 접촉하거나 공격 범위에 들어오면 전투
아이템	포션(체력 회복), 무기(공격력 증가) 등 2~3종
층 구조	출구로 나가면 다음 층. 예: 1~2층은 일반 몬스터, 마지막 층에 보스
보스	체력이 높고 공격이 강한 몬스터. 이걸 잡으면 승리
패배	체력이 0이 되면 게임 종료(로그라이크의 영구 죽음)

3.2 단계적으로 키우는 계획 (한 번에 다 만들지 않기)

아래처럼 3단계로 나눠 점점 복잡하게 만듭니다. 각 단계가 '학습이 되는지' 확인된 뒤에만 다음 단계로 넘어갑니다.

- 1단계(가장 단순): 맵 하나 고정, 적 1~2마리, 아이템 없음. 목표는 출구 도달. → AI가 길 찾기를 배우는지 확인.
- 2단계: 맵을 매번 무작위로 생성, 적과 아이템 추가, 여러 층. → AI가 상황 판단(싸울지 피할지, 포션 언제 쓸지)을 배우는지 확인.
- 3단계(목표): 마지막 층에 보스 배치. → AI가 자원을 아껴 보스전까지 살아남고 보스를 처치하는지 확인.

3.3 화면 표현

처음엔 터미널에 글자로 그리는 것(예: 벽은 #, 나는 @, 적은 E, 보스는 B)만으로 충분합니다. 학습에는 그래픽이 필요 없습니다. 나중에 여유가 있으면 pygame으로 간단한 2D 화면을 붙일 수 있지만, 이는 선택 사항이며 로드맵 후반의 보너스로 둡니다.

4. 기술 스택 (사용할 도구)

입문자에게는 '직접 다 만들기'보다 '검증된 도구를 조립하기'가 훨씬 낫습니다. 아래 도구들은 무료이고 자료가 많으며, 강화학습 입문의 사실상 표준입니다.

도구	역할	왜 이것을
Python	전체 개발 언어	입문자 친화적이고 AI 분야 표준. 지금 배우는 언어 그대로 사용
Gymnasium	게임(환경)을 강화학습이 이해하는 형식으로 감싸는 규격	우리 게임과 AI를 연결하는 표준 '콘센트' 역할
Stable-Baselines3	PPO 등 강화학습 알고리즘이 완성된 라이브러리	알고리즘을 직접 안 만들어도 됨. 몇 줄로 학습 시작 가능
NumPy	숫자/배열 계산	게임 상태를 숫자로 다루는 데 필수, 기본 도구
Matplotlib	그래프 그리기	학습 점수가 오르는 곡선을 그려 성과를 눈으로 확인
(선택) pygame	간단한 2D 화면	후반에 여유가 있으면 시각화용으로만 추가

설치는 나중에 로드맵 1주차에서 한 줄 명령으로 진행합니다. 지금은 '이런 도구들을 쓴다' 정도만 기억하면 됩니다. 참고로 Gymnasium과 Stable-Baselines3는 원래 게임 AI 연구에서 쓰던 도구라, 우리 프로젝트에 딱 맞습니다.

5. 시스템 구조 (전체 그림)

프로젝트는 크게 세 덩어리로 이루어집니다. 각 덩어리를 따로 만들고 나중에 연결합니다.

구성 요소	하는 일	쉽게 말하면
게임 엔진	로그라이크 규칙을 코드로 구현	'놀이터'를 만드는 부분

구성 요소	하는 일	쉽게 말하면
학습 환경(Gymnasium 래퍼)	게임을 AI가 주고받을 수 있는 형식으로 변환	놀이터와 AI 사이의 '통역사'
학습 에이전트(PPO)	반복 플레이하며 정책(policy)을 학습	실제로 배우는 'AI 두뇌'
전술 추천기	학습된 두뇌에게 상황을 물어 답을 사람 말로 변환	AI가 배운 걸 사람에게 '설명'하는 부분

데이터 흐름: 게임 엔진이 현재 상태를 통역사(환경)에게 주면, 통역사가 AI 두뇌에게 전달합니다. AI 두뇌가 행동을 정하면 다시 통역사를 거쳐 게임 엔진이 그 행동을 실행하고 보상을 돌려줍니다. 이 고리가 반복되면서 AI가 학습합니다. 학습이 끝난 뒤에는 전술 추천기가 이 두뇌를 불러와, 사용자가 준 상황에 대해 '어떤 행동을 추천하는지' 물어봅니다.

폴더 구조 예시(참고용):

- game/ — 게임 규칙 코드(맵 생성, 전투, 아이템)
- env/ — Gymnasium 환경 래퍼(게임↔AI 연결)
- train/ — 학습 실행 스크립트, 보상 설정, 저장된 모델
- recommend/ — 전술 추천 기능
- results/ — 그래프, 영상, 로그

6. 3개월 실행 로드맵 (주 단위)

총 12주 계획입니다. 하루 1~2시간 기준으로 짰고, 각 주는 '이번 주에 끝내면 되는 것' 하나에 집중합니다. 진도가 밀려도 괜찮습니다. 뒤에 나오는 '완충 주간'이 있어 조정할 수 있습니다. 각 달의 목표를 먼저 기억하세요: 1개월차=기초와 게임 만들기, 2개월차=AI가 배우게 하기, 3개월차=보스 공략과 추천 기능.

6.1 1개월차 — Python 기초 다지기 + 게임 만들기

이 달의 목표: 사람이 직접 플레이할 수 있는 간단한 로그라이크 게임을 완성한다. 아직 AI는 등장하지 않습니다. AI를 붙이기 전에 '놀이터'부터 튼튼하게 만드는 단계입니다.

주차	할 일	이번 주 완료 기준
1주차	개발 환경 설치(Python, 편집기), 도구 설치(NumPy 등). Python 기초 복습: 변수, 조건문, 반복문, 함수, 리스트	화면에 격자 맵을 글자로 출력할 수 있다
2주차	플레이어 이동 구현: 키 입력으로 상하좌우 움직이기, 벽 충돌 처리, 체력(HP) 개념	키를 눌러 맵 위 캐릭터를 움직일 수 있다
3주차	적과 전투 구현: 몬스터 배치, 공격, 체력 감소, 죽음 처리. 아이템(포션·무기) 추가	적과 싸우고 아이템을 먹을 수 있다
4주차	층 구조와 보스 뼈대: 출구로 다음 층 이동, 무작위 맵 생성, 마지막 층 보스 배치. 사람이 끝까지 플레이 가능하게 정리	사람이 직접 보스까지 도달해 승패가 갈린다

6.2 2개월차 — AI가 게임을 배우게 만들기

이 달의 목표: 앞서 만든 게임을 AI가 플레이하고 학습하도록 연결한다. 여기서 처음으로 강화학습이 등장합니다. 핵심은 '게임을 AI 형식으로 감싸기(환경 만들기)'와 '보상 설계'입니다.

주차	할 일	이번 주 완료 기준
5주차	Gymnasium 학습, 게임을 환경으로 감싸기: 상태(state)를 숫자로 표현, 행동(action) 목록 정의	AI가 무작위로라도 게임을 자동 플레이한다

주차	할 일	이번 주 완료 기준
6주차	보상 설계 1차: 적 처치 +, 피해 -, 층 진행 +, 죽음 - 등 점수 규칙 정하기	게임이 끝날 때 총점이 계산되어 나온다
7주차	Stable-Baselines3로 PPO 학습 시작: 짧게 학습 후 점수 곡선 확인	학습을 돌리면 점수가 오르기 시작한다(1단계 맵 기준)
8주차	학습 안정화 + 보상 재조정: 이상 행동(구석에 숨기 등) 잡기, 학습 설정 조정, 결과 그래프화	AI가 1~2층을 안정적으로 통과한다

6.3 3개월차 — 보스 공략 + 전술 추천

이 달의 목표: AI가 보스를 처치하도록 만들고, 배운 전술을 사람에게 추천하는 기능을 완성한다. 프로젝트를 마무리하고 결과물을 정리하는 단계입니다.

주차	할 일	이번 주 완료 기준
9주차	보스전 학습 집중: 3단계 맵(보스 포함)으로 학습, 자원 관리(포션 아끼기)를 배우도록 보상 조정	AI가 가끔이라도 보스를 처치한다
10주차	성능 끌어올리기: 더 오래 학습, 승률 측정, 여러 번 돌려 안정적으로 보스 처치	보스 처치 승률이 눈에 띄게 오른다
11주차	전술 추출 + 추천 기능(7장): 학습된 AI에게 상황을 물어 추천 행동을 사람 말로 변환	상황을 입력하면 추천 전술이 문장으로 나온다
12주차	마무리: 플레이 영상/GIF 만들기, 점수 곡선 정리, 코드 정리와 사용 설명서 작성, 회고	결과물 일체가 정리되어 남에게 보여줄 수 있다

6.4 완충 주간에 대하여

위 12주는 이상적인 경우입니다. 입문자는 특정 주에서 막히기 마련이므로, 실제로는 각 달의 마지막 주(4·8·12주차)를 '못다 한 일 따라잡기 + 정리' 시간으로 유연하게 쓰세요. 진도가 밀리면 게임 규칙을 더 단순하게 줄이는 것이 정답입니다. 규모를 키우는 건 시간이 남을 때만 합니다.

7. 전술 추천 시스템 설계

이 부분은 프로젝트의 마지막 요구사항이자 차별점입니다. '학습으로 배운 수많은 전술을 사용자에게 맞게 추천'하는 기능입니다. 어렵게 들리지만, 핵심 아이디어는 단순합니다: 학습이 끝난 AI 두뇌는 이미 '어떤 상황에서 무엇을 해야 하는지'를 알고 있습니다. 그 두뇌에게 상황을 물어보고, 대답을 사람이 알아들을 수 있는 문장으로 바꿔주면 됩니다.

7.1 작동 방식(3단계)

5. 상황 입력: 사용자가 현재 게임 상황을 알려줍니다(예: 체력 20%, 앞에 보스, 포션 1개 보유). 실제 플레이 중 자동으로 읽어올 수도 있습니다.
6. AI에게 질문: 학습된 AI 두뇌에게 이 상황을 주고 '가장 좋은 행동'과 '각 행동의 선호도'를 받아옵니다.
7. 사람 말로 변환: '포션 사용(추천도 80%), 후퇴(15%), 공격(5%)' 같은 결과를 '지금은 체력이 낮으니 포션을 먼저 쓰고 거리를 두세요' 같은 문장으로 바꿔 보여줍니다.

7.2 '사용자에게 맞게' 추천한다는 것

사용자마다 플레이 성향이 다릅니다. 이를 반영하는 방법을 난이도 순서로 준비합니다. 쉬운 방법부터 시작하고, 시간이 남으면 다음 단계로 확장하세요.

- 기본(반드시 구현): 현재 상황에 대해 AI가 추천하는 행동과 그 이유를 알려준다.
- 중급(가능하면): 스타일 옵션을 준다. 예를 들어 '공격적' 성향이면 위험을 감수하는 행동에, '안전' 성향이면 생존 위주 행동에 가중치를 준다.
- 심화(여유 시): 여러 번의 학습 플레이 기록을 분석해 '보스전에서 이긴 판들은 이런 공통 패턴이 있었다'처럼 전술 요약을 뽑아 추천에 활용한다.

7.3 결과물 형태

추천 기능은 처음엔 간단한 명령줄 프로그램으로 충분합니다. 상황을 입력하면 추천 문장이 출력되는 형태입니다. 나중에 여유가 있으면 게임 화면 옆에 실시간 조언을 띄우는 형태로 발전시킬 수 있지만, 이는 보너스입니다.

8. 예상되는 어려움과 대응

입문자가 자주 막히는 지점을 미리 알아두면 당황하지 않습니다. 대부분의 문제는 '규모를 줄이는 것'으로 해결됩니다.

어려움	증상	대응
AI가 안 배움	학습해도 점수가 안 오름	맵/규칙을 더 단순하게, 보상을 더 명확하게. 1단계 맵부터 다시 확인
엉뚱한 행동	구석에 숨거나 목표를 무시	보상 설계 재검토. '목표 접근'에 + 점수, '시간 끌기'에 -점수 추가
학습이 느림	결과 보는 데 너무 오래 걸림	맵 크기 축소, 학습 단계를 짧게 여러 번 나눠 확인
Python 문법 막힘	코드 오류가 잦음	AI가 학습하기 전 단계이므로 여유 있음. 오류 메시지를 그대로 검색/질문
일정 지연	주차 계획이 밀림	완충 주간 활용. 기능을 줄여서라도 '동작하는 완성본' 우선

9. 성공 기준

프로젝트가 성공했다고 말할 수 있는 기준을 세 단계로 둡니다. 최소 기준만 넘겨도 '강화학습을 표현한다'는 목표는 달성입니다.

단계	기준
최소 성공(반드시)	게임이 완성되고, AI가 학습으로 점수를 올려 최소 1~2층을 안정적으로 통과한다. 점수 곡선으로 학습을 증명한다
목표 성공	AI가 보스를 어느 정도 승률로 처치하고, 기본 전술 추천 기능이 동작한다
도전 성공	높은 보스 처치 승률, 사용자 성향을 반영한 추천, 시각화된 플레이 영상까지 완성한다

10. 지금 바로 할 일 (다음 단계)

이 문서를 다 읽었다면, 다음 순서로 시작하세요. 한 번에 하나씩만 하면 됩니다.

8. 개발 환경 준비: Python과 코드 편집기(예: VS Code)를 설치한다. (1주차 시작)
9. Python 기초 훑기: 변수·조건문·반복문·함수·리스트를 짧게 복습한다.
10. 가장 작은 게임부터: 격자 맵을 글자로 출력하는 코드 한 개를 완성해 본다.
11. 막히면 이 문서로 돌아와 해당 장을 다시 읽는다. 특히 6장 로드맵과 2장 개념.

필요하면 각 주차별로 더 자세한 실습 가이드나 예제 코드, 학습 자료 목록을 이어서 만들어 드릴 수 있습니다. 이 설계 문서를 나침반으로 삼아 한 주씩 나아가면 됩니다.